



Pro-code Statistical Visualization with Seaborn

DSCI 454
Week 10, Lecture 2



What are we learning this lecture?

- Pro-code Statistical Visualization with Seaborn
 - Axes vs Figure-level functions
 - relplot, displot, catplot
 - Customizable Abstractions
 - FacetGrid
 - PairGrid
 - JointGrid



Multi-plot grid for plotting conditional relationships.

This class maps a dataset onto multiple axes arrayed in a grid of rows and columns that correspond to *levels* of variables in the dataset. The plots it produces are often called “lattice”, “trellis”, or “small-multiple” graphics.

It can also represent levels of a third variable with the `hue` parameter, which plots different subsets of data in different colors. This uses color to resolve elements on a third dimension, but only draws subsets on top of each other and will not tailor the `hue` parameter for the specific visualization the way that axes-level functions that accept `hue` will.

The basic workflow is to initialize the `FacetGrid` object with the dataset and the variables that are used to structure the grid. Then one or more plotting functions can be applied to each subset by calling `FacetGrid.map()` or `FacetGrid.map_dataframe()`. Finally, the plot can be tweaked with other methods to do things like change the axis labels, use different ticks, or add a legend. See the detailed code examples below for more information.

<https://seaborn.pydata.org/generated/seaborn.FacetGrid.html>

```
g = sns.FacetGrid(tips, col="sex",  
row="time", margin_titles=True)
```

```
g.map_dataframe(sns.scatterplot,  
x="total_bill", y="tip",  
hue='smoker')
```

```
g.set_axis_labels("Total bill ($)",  
"Tip ($)")  
g.set_titles(col_template="{col_name}  
patrons", row_template="{row_name}")  
g.set(xlim=(0, 60), ylim=(0, 12),  
xticks=[10, 30, 50], yticks=[2, 6,  
10])  
g.add_legend(title='Smoker')  
g.tight_layout()  
g.savefig("facet_plot.png")
```

FacetGride output

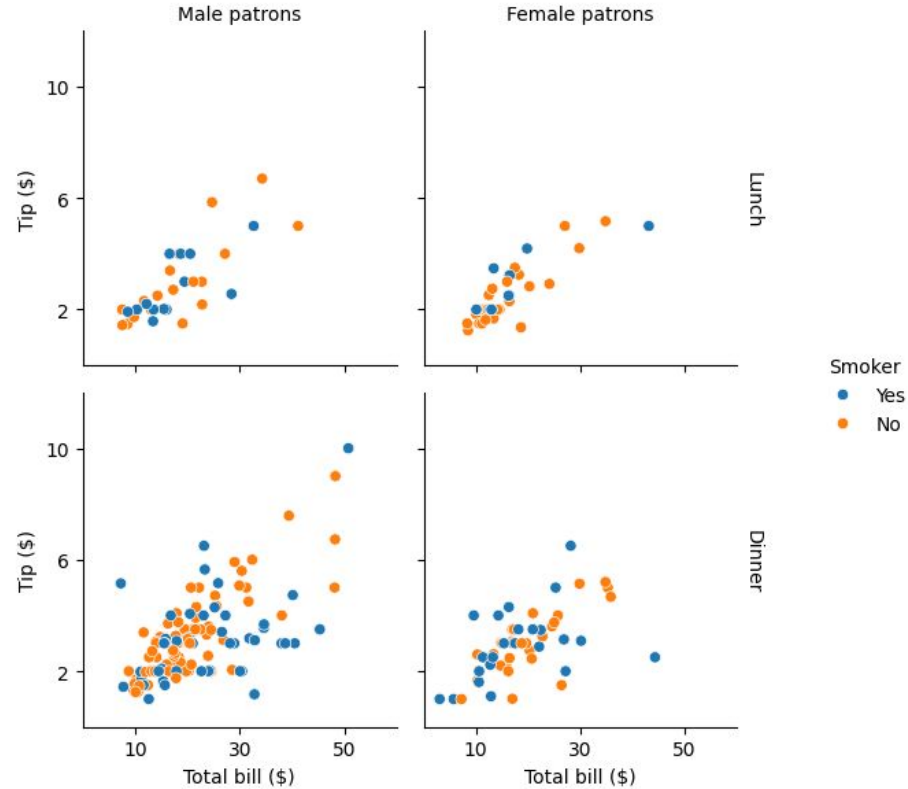


Figure-level interface for drawing categorical plots onto a FacetGrid.

This function provides access to several axes-level functions that show the relationship between a numerical and one or more categorical variables using one of several visual representations. The `kind` parameter selects the underlying axes-level function to use.

Extra keyword arguments are passed to the underlying function, so you should refer to the documentation for each to see kind-specific options.

<https://seaborn.pydata.org/generated/seaborn.catplot.html>

Categorical scatterplots:

- `stripplot()` (with `kind="strip"`; the default)
- `swarmplot()` (with `kind="swarm"`)

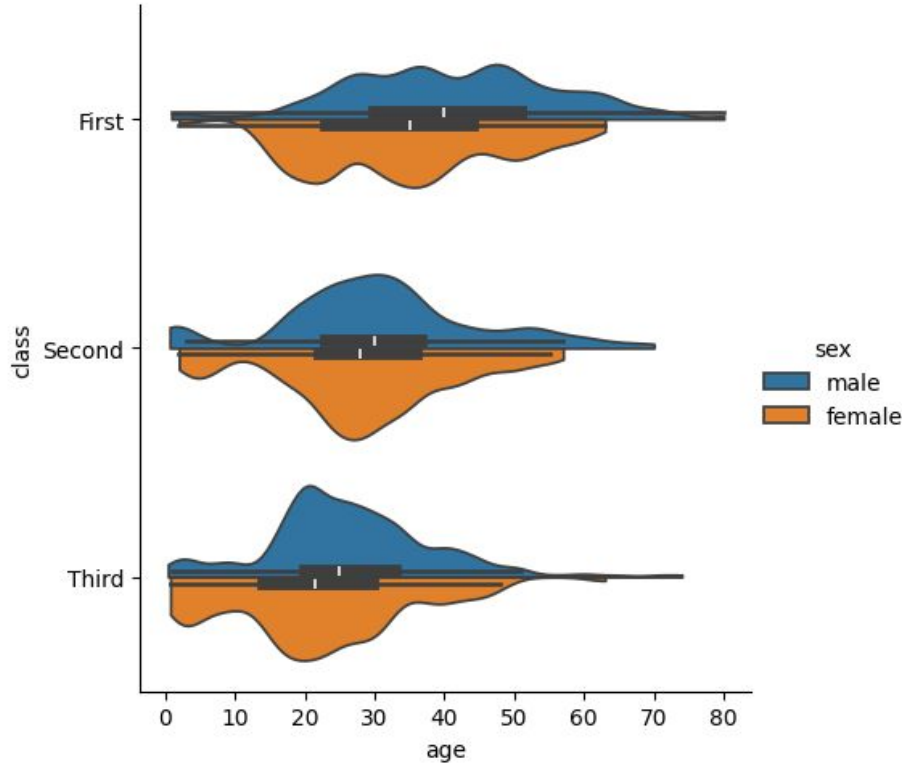
Categorical distribution plots:

- `boxplot()` (with `kind="box"`)
- `violinplot()` (with `kind="violin"`)
- `boxenplot()` (with `kind="boxen"`)

Categorical estimate plots:

- `pointplot()` (with `kind="point"`)
- `barplot()` (with `kind="bar"`)
- `countplot()` (with `kind="count"`)

catplot



```
g=sns.catplot(  
    data=df, x="age", y="class", hue="sex",  
    kind="violin", bw_adjust=.5, cut=0,  
    split=True  
)
```

Figure-level interface for drawing relational plots onto a FacetGrid.

This function provides access to several different axes-level functions that show the relationship between two variables with semantic mappings of subsets. The `kind` parameter selects the underlying axes-level function to use:

- `scatterplot()` (with `kind="scatter"`; the default)
- `lineplot()` (with `kind="line"`)

The relationship between `x` and `y` can be shown for different subsets of the data using the `hue`, `size`, and `style` parameters. These parameters control what visual semantics are used to identify the different subsets. It is possible to show up to three dimensions independently by using all three semantic types, but this style of plot can be hard to interpret and is often ineffective. Using redundant semantics (i.e. both `hue` and `style` for the same variable) can be helpful for making graphics more accessible.

The default treatment of the `hue` (and to a lesser extent, `size`) semantic, if present, depends on whether the variable is inferred to represent “numeric” or “categorical” data. In particular, numeric variables are represented with a sequential colormap by default, and the legend entries show regular “ticks” with values that may or may not exist in the data. This behavior can be controlled through various parameters, as described and illustrated below.

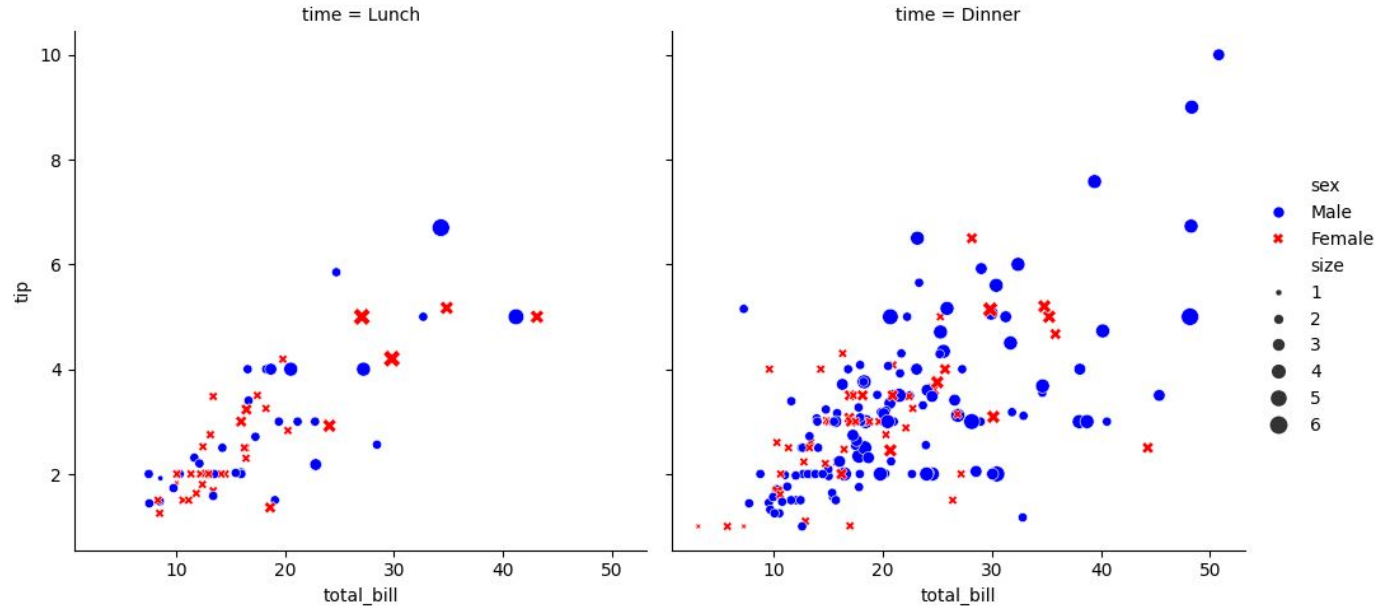
After plotting, the `FacetGrid` with the plot is returned and can be used directly to tweak supporting plot details or add other layers.

<https://seaborn.pydata.org/generated/seaborn.relplot.html>

relplot



```
g=sns.relplot(  
    data=tips, x="total_bill", y="tip", col="time", hue="sex", size="size", style="sex",  
    palette=["b", "r"], sizes=(10, 100)  
)
```



displot (not disTplot (prev. version))



Figure-level interface for drawing distribution plots onto a FacetGrid.

This function provides access to several approaches for visualizing the univariate or bivariate distribution of data, including subsets of data defined by semantic mapping and faceting across multiple subplots. The `kind` parameter selects the approach to use:

- `histplot()` (with `kind="hist"`; the default)
- `kdeplot()` (with `kind="kde"`)
- `ecdfplot()` (with `kind="ecdf"`; univariate-only)

Additionally, a `rugplot()` can be added to any kind of plot to show individual observations.

Extra keyword arguments are passed to the underlying function, so you should refer to the documentation for each to understand the complete set of options for making plots with this interface.

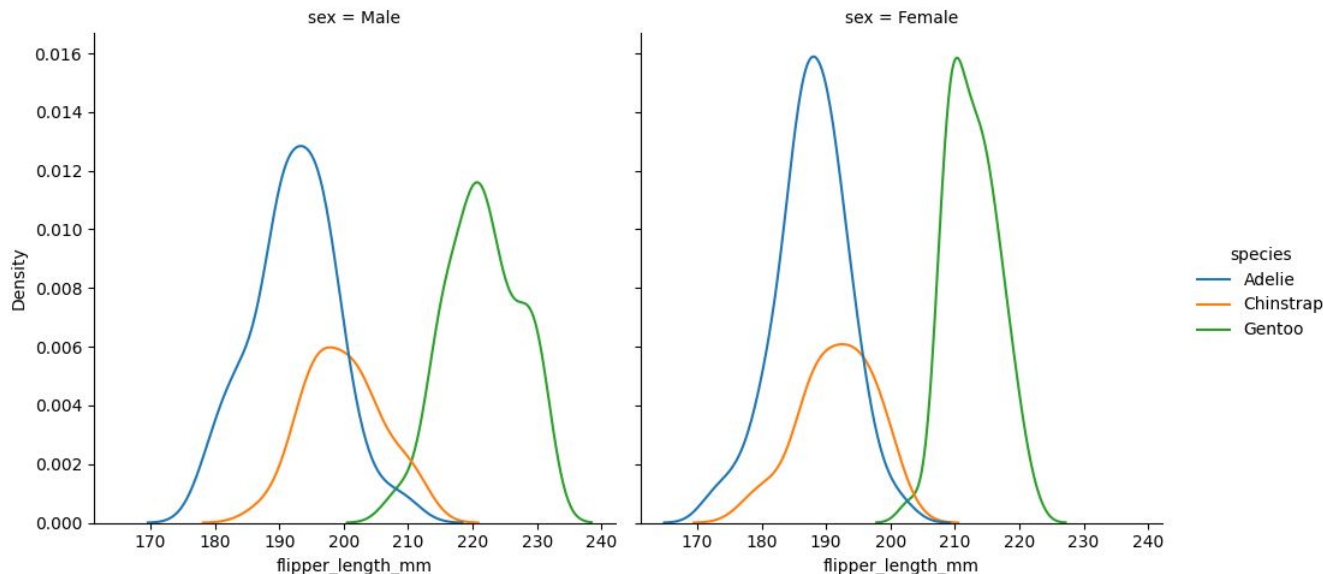
See the [distribution plots tutorial](#) for a more in-depth discussion of the relative strengths and weaknesses of each approach. The distinction between figure-level and axes-level functions is explained further in the [user guide](#).

<https://seaborn.pydata.org/generated/seaborn.displot.html>

displot (not disTplot)



```
g=sns.displot(data=penguins, x="flipper_length_mm",  
              hue="species", col="sex", kind="kde")
```



Implot



Plot data and regression model fits across a FacetGrid.

This function combines `regplot()` and `FacetGrid`. It is intended as a convenient interface to fit regression models across conditional subsets of a dataset.

When thinking about how to assign variables to different facets, a general rule is that it makes sense to use `hue` for the most important comparison, followed by `col` and `row`. However, always think about your particular dataset and the goals of the visualization you are creating.

There are a number of mutually exclusive options for estimating the regression model. See the [tutorial](#) for more information.

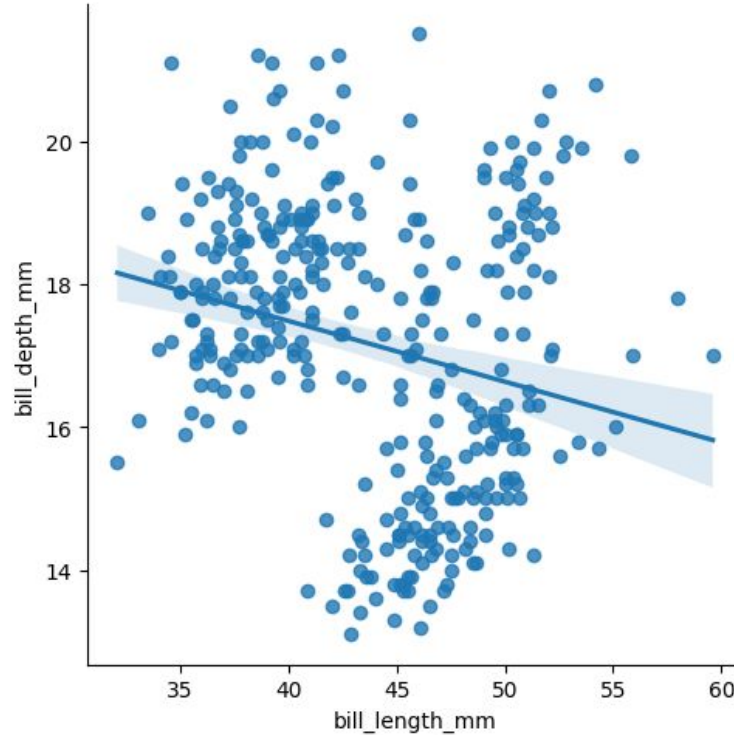
The parameters to this function span most of the options in `FacetGrid`, although there may be occasional cases where you will want to use that class and `regplot()` directly.

<https://seaborn.pydata.org/generated/seaborn.Implot.html>

Implot



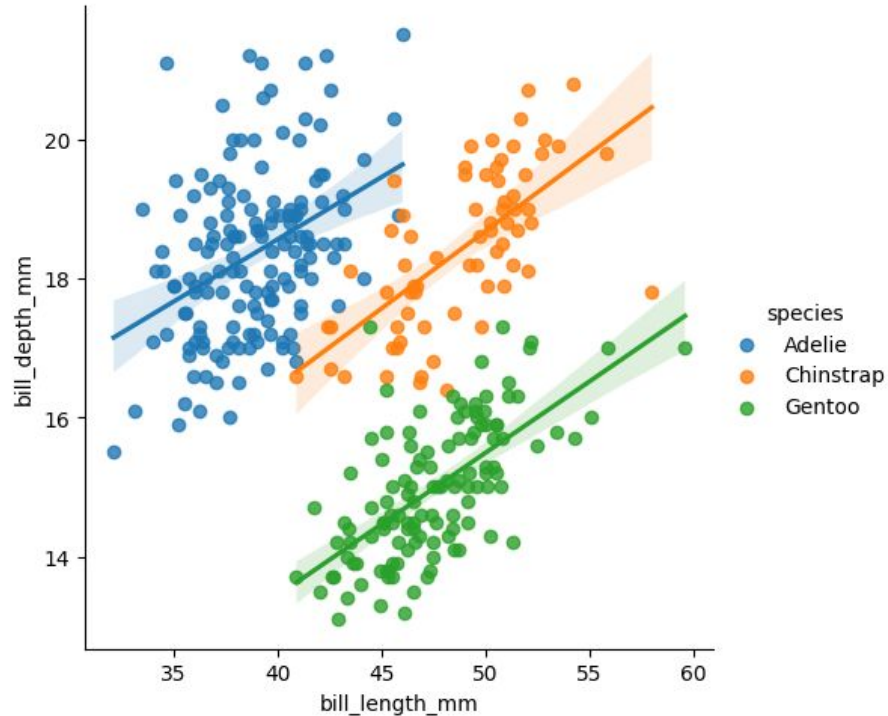
```
sns.lmplot(data=penguins, x="bill_length_mm", y="bill_depth_mm")
```



Implot



```
sns.lmplot(data=penguins, x="bill_length_mm", y="bill_depth_mm", hue="species")
```



[Simpson's paradox](#) is a phenomenon in probability and statistics in which a trend appears in several groups of data but disappears or reverses when the groups are combined.

PairGrid



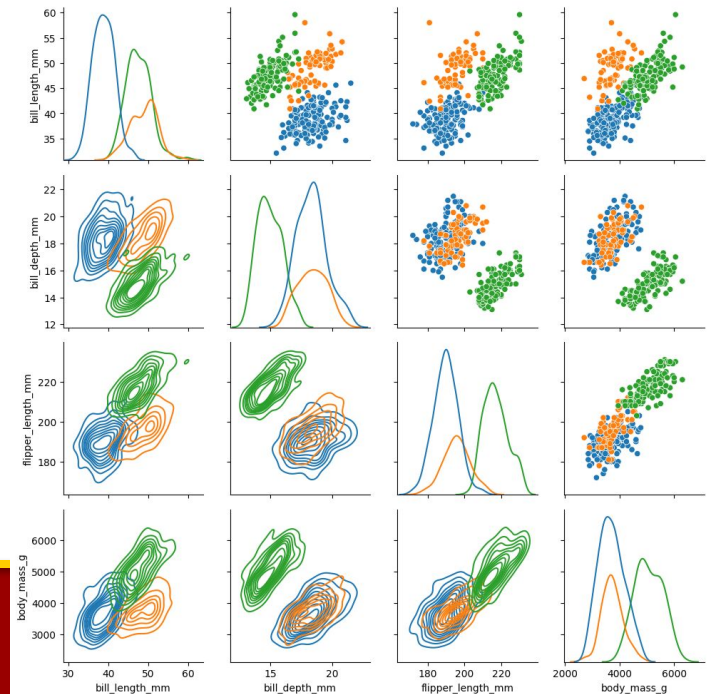
Subplot grid for plotting pairwise relationships in a dataset.

This object maps each variable in a dataset onto a column and row in a grid of multiple axes. Different axes-level plotting functions can be used to draw bivariate plots in the upper and lower triangles, and the marginal distribution of each variable can be shown on the diagonal.

Several different common plots can be generated in a single line using `pairplot()`. Use `PairGrid` when you need more flexibility.

<https://seaborn.pydata.org/generated/seaborn.PairGrid.html>

```
g = sns.PairGrid(penguins,  
diag_sharey=False, hue='species')  
g.map_upper(sns.scatterplot)  
g.map_lower(sns.kdeplot)  
g.map_diag(sns.kdeplot)
```



JointGrid

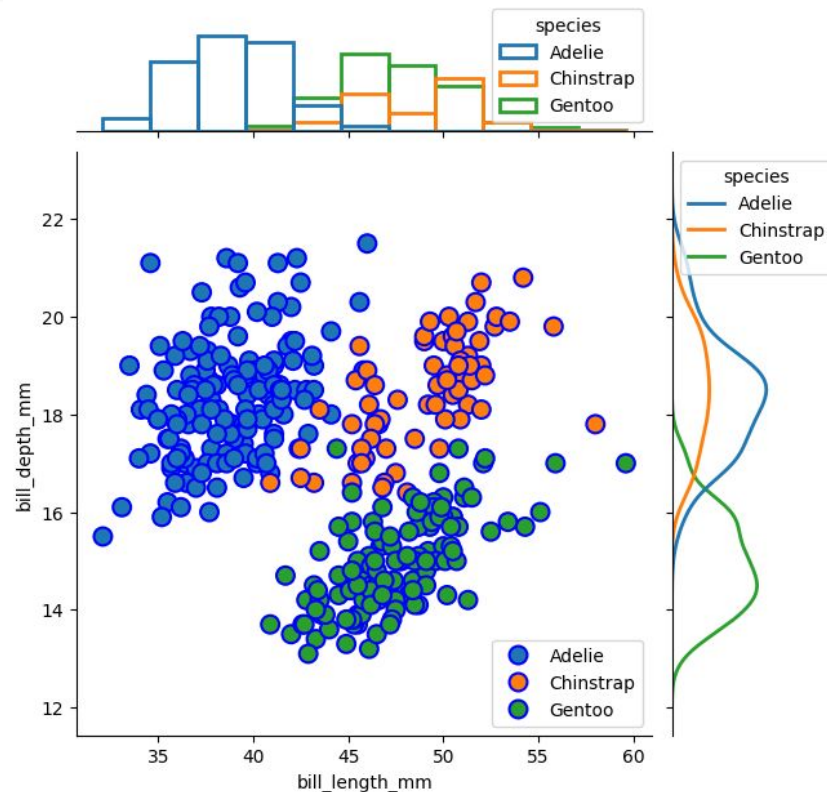


Grid for drawing a bivariate plot with marginal univariate plots.

Many plots can be drawn by using the figure-level interface `jointplot()`. Use this class directly when you need more flexibility.

<https://seaborn.pydata.org/generated/seaborn.JointGrid.html>

```
g = sns.JointGrid()
x, y =
penguins["bill_length_mm"],
penguins["bill_depth_mm"]
sns.scatterplot(x=x, y=y, ec="b",
s=100, linewidth=1.5,
ax=g.ax_joint,
hue=penguins['species'])
g.ax_joint.legend(loc='lower
right')
sns.histplot(x=x, fill=False,
linewidth=2, ax=g.ax_marg_x,
hue=penguins['species'])
sns.kdeplot(y=y, linewidth=2,
ax=g.ax_marg_y,
hue=penguins['species'])
```



Resources



- [seaborn's own tutorial](#)
- [Dr. Kimberly Fessel's Intro to seaborn YT Playlist](#)

Questions?

